1 Introduction to Software Engineering Management

Week 1 · Week summary

Core ideas, lesson by lesson

1 What software engineering is

- Software engineering is a discipline about delivering and maintaining software, not only writing it.
- Most project problems come from communication and planning gaps.
- Models let teams discuss and check a system before building it.

2 The development lifecycle and process models

- A process model is a simplified description of how real teams work.
- Iterative work produces baselines; the difference between baselines is an increment.
- Choose a process by how much change you expect and how costly late mistakes are.

3 Software configuration management

- Configuration items include everything that makes up the product, not only code.
- SCM answers: which version, which change, approved by whom, and in which baseline.

4 Version control systems

- A VCS stores history, supports recovery, and shows who changed what and when.
- Git is distributed; GitHub is a hosting service for Git repositories.
- One commit should hold one logical change with a clear message.

5 Git fundamentals: areas, commits, and branches

- Edit in the working directory, stage with git add, and record with git commit.
- git status and git log show where you are; use them before and after each step.
- A branch is a pointer. Merging combines history; conflicts require a human decision.

6 Git hosting, remotes, and collaboration workflows

- origin is the conventional name for the remote you cloned or connected.
- fetch downloads; pull downloads and integrates; push uploads.
- Pull requests make review a planned step before code reaches main.

Watch out

Mistake	Fix
Confusing Git with GitHub.	Git is the tool on your machine; GitHub hosts repositories.
Forgetting to read git status before committing.	Make git status a habit before and after every commit.

Treating "it compiles" as "it meets the need".	Compare the result with written requirements and tests.
Calling a waterfall project "agile" because it has meetings.	Ask whether feedback changes the plan during the project.
Writing commit messages like "stuff" or "changes".	Describe the change in an imperative line of about 50 characters.
Committing without checking git status, so unrelated files get included.	Run git status and git diff --staged before each commit.
Deleting conflict markers without choosing the right content.	Decide the final content for the lines, remove all three marker lines, then stage and commit.
Pushing directly to main on a shared project.	Work on a feature branch and open a pull request for review.